

Understanding Python's `def` Statement

A complete reference guide on defining functions, argument parsing, scope, decorators, and execution internals.

1. Introduction & Syntax Overview

In Python, the `def` keyword is an executable statement used to create user-defined function objects. Unlike compiled languages where function declarations are processed at compile-time, in Python `def` executes at runtime to bind a code block to a specific variable name in the current scope.

```
def function_name(param1, param2: int = 0) -> bool:
    """Optional docstring describing what the function does."""
    # Function body
    return True
```

2. Parameter Types & Argument Parsing

Python provides flexible parameter definitions, allowing positional, keyword, default, and variable-length arguments. Python 3.8+ introduced explicit positional-only and keyword-only markers:

```
def complex_func(p_only, /, standard, *, k_only="default", **kwargs):
    pass
```

Syntax Category	Example Definition	Description
Positional / Keyword	<code>def f(x, y):</code>	Arguments can be passed by position or explicitly named.
Positional-Only (<code>/</code>)	<code>def f(x, /):</code>	Arguments before <code>/</code> MUST be passed positionally.
Keyword-Only (<code>*</code>)	<code>def f(*, key):</code>	Arguments after <code>*</code> MUST be explicitly named when called.
Variable Positional	<code>def f(*args):</code>	Packs remaining positional arguments into a <code>tuple</code> .
Variable Keyword	<code>def f(**kwargs):</code>	Packs remaining keyword arguments into a <code>dict</code> .

⚠ Common Trap: Mutable Default Arguments

Default values are evaluated **once when the function is defined**, not when it is called! Passing mutable objects like lists or dictionaries as defaults persists state across calls.

Bad: `def append_to(element, target=[]):`

Good: `def append_to(element, target=None): if target is None: target = []`

3. Scoping & The LEGB Rule

Variables created inside a `def` block are local to that function. When evaluating variable names inside a function, Python searches four scopes sequentially (the **LEGB** rule):

- **L — Local:** Names defined within the current function body (or lambda).
- **E — Enclosing:** Names in local scopes of enclosing/nested functions (closures).
- **G — Global:** Names defined at the top level of the current module file.
- **B — Built-in:** Predefined names in Python's standard library (e.g., `len`, `range`).

```
x = "Global"

def outer():
    x = "Enclosing"
    def inner():
        nonlocal x # Modifies outer's 'x' variable
        x = "Modified Enclosing"
    inner()
    print(x) # Output: Modified Enclosing
```

4. How `def` Works Under the Hood

When Python's interpreter executes a `def` block, it performs these steps:

1. **Bytecode Compilation:** Compiles the body of the function into a bytecode object (stored in `fn.__code__`).
2. **Default Argument Storage:** Evaluates default expressions once and stores them in `fn.__defaults__` or `fn.__kwdefaults__`.
3. **Function Object Creation:** Instantiates a runtime function object wrapping code, defaults, global scope reference (`fn.__globals__`), and docstrings.
4. **Variable Binding:** Binds the function object to the function name in the local namespace.

```
def add(a, b=10):
    return a + b

print(add.__code__) # <code object add at ...>
print(add.__defaults__) # (10, )
print(add.__name__) # 'add'
```

5. First-Class Functions & Decorators

In Python, functions defined via `def` are **first-class citizens**: they can be assigned to variables, passed as arguments, returned from other functions, and stored in data structures.

Decorators Syntactic Sugar

A decorator is a higher-order function that takes a function object as an input and returns a modified function object. The `@decorator` syntax is syntactic sugar for wrapping a definition.

```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Calling function: {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

# Using decorator syntax
@logger
def greet(name):
    return f"Hello, {name}!"

# Equivalent to: greet = logger(greet)
print(greet("Alice"))
```

6. Comparison: `def` VS `lambda`

Feature	<code>def</code> Statement	<code>lambda</code> Expression
Statement Type	Standard multi-line statement	Single-line inline expression
Return Behavior	Requires explicit <code>return</code> (or returns <code>None</code>)	Implicitly returns the evaluated expression
Docstrings & Type Hints	Fully supported	Not supported
Reusability	Designed for reusable, complex logic	Best for quick throwaway callbacks (e.g. <code>sorted()</code>)